Magpie — Sprint 8 Technical Spec (Durability)

Audience: implementing agent (Codex) + Regan. **Builds on** Sprints 0–7. **Status: planning / review — implement after approval.**

> This sprint makes the current system **survivable**: an automated, encrypted, offsite backup of the live data, plus a **tested restore** and rotating no-secret logs. It does **not** change how Magpie runs (that is Sprint 9), does **not** migrate the database, does **not** add HTTPS, and does **not** touch application features.

Confirmed environment (F-14 — do not re-assume): live DB is **SQLite** (`…\backend\db.sqlite3` , ~553 KB) — *not* Postgres (CI-only). Host is **Windows 10 / PowerShell 7.4**. Media root is `…\backend\media_files` . Real data present: 8 items, 2 photos, 5 valuations, 2 drafts, 1 eBay credential. **The backup targets SQLite + media on Windows.**

---

0. Locked decisions this spec implements

D-76 (split + this is the SQLite/Windows-confirmed Sprint 8), D-77 (backup contents + mechanism), D-78 (out-of-band secrets, never in-archive), D-79 (closure gate), D-80 (rotating logs + secret-grep), D-81 (HTTPS/settings deferred to Sprint 9).

---

1. What gets built

1. An extended `python manage.py backup` that produces one encrypted, timestamped archive containing the live SQLite DB, the media tree, an env-var **name** manifest, and a restore runbook — then optionally uploads it offsite.
2. A `python manage.py restore` that decrypts + restores an archive onto a clean location (the other half of "a backup you've tested").
3. **Rotating application logging** to files, with the structural no-secret discipline.
4. A **Windows Task Scheduler** registration (daily, unattended) + a PowerShell wrapper.
5. Tests (run in CI on SQLite) + the operational `RESTORE_DRILL_EVIDENCE.md` .

No new app. Work lives in an existing/ops module (e.g. `apps/core/management/commands/`). **One consideration on dependencies:** encryption reuses the already-present `cryptography` library (no new binary to install on Windows); cloud upload uses `rclone` (a portable Windows binary — the one new operational tool, provider-agnostic).

---

2. The backup command (`manage.py backup`)

First, inspect the existing command (the project has had `manage.py backup` / `restore` since Sprint 0, extended per sprint). Keep its table-coverage intent; this spec defines the *authoritative* mechanism below and supersedes any Postgres-specific path.

Pipeline (all pure-Python so it stays cross-platform for a future Linux move):

1. **Consistent SQLite snapshot — not a raw file copy.** Use the SQLite **online backup API** so the snapshot is consistent even if the app is mid-write (handles WAL):

   ```
   import sqlite3
   src = sqlite3.connect(db_path)                  # the live db.sqlite3
   dst = sqlite3.connect(str(snapshot_path))       # temp file in the work dir
   with dst:
       src.backup(dst)                             # atomic, consistent
   src.close(); dst.close()
   ```

   This is the **authoritative** DB backup (more faithful than `dumpdata`). The snapshot file is what `restore` puts back.

2. **Media tree** — copy `MEDIA_ROOT` (`media_files`) into the work dir with `shutil.copytree` (preserves originals/processed/thumbnails). Defensive: skip nothing, follow the real tree.

3. **Env-var name manifest** — write `env_manifest.json` listing the **names** of required env vars (e.g. `DATABASE_URL`, `MAGPIE_TOKEN_ENCRYPTION_KEY`, `EBAY_ENV`, `EBAY_CLIENT_ID`, `EBAY_CLIENT_SECRET`, `EBAY_RU_NAME`, `EBAY_SCOPES`, `MAGPIE_BACKUP_PASSPHRASE`) — **names only, never values** (test-asserted).

4. **Restore runbook** — copy a bundled `RESTORE_RUNBOOK.md` (step-by-step recovery, incl. the out-of-band-secrets reminder and the eBay reconnect note).

5. **Archive** — tar+gzip the work dir → `magpie-backup-YYYYMMDD-HHMMSS.tar.gz` (Python `tarfile`).

6. **Encrypt** (§3) → `magpie-backup-YYYYMMDD-HHMMSS.tar.gz.enc` ; delete the plaintext tar and the work dir.
7. **Output** to a local backups directory (default `…\backend\backups\` , configurable); **prune** per retention (§6).
8. **Upload** (if `--upload`): `rclone copy <archive> <remote>:<path>` to the configured cloud remote.

Flags: `--output-dir PATH` , `--upload/--no-upload` (default no-upload for manual runs; the scheduled task passes `--upload`), `--keep-daily N --keep-weekly M` .

Never log the passphrase, the token key, any env *value*, or the eBay credential. Log only filenames, sizes, counts, and the upload destination name.

---

3. Archive encryption (`cryptography` -native — no new install)

Reuse the in-project `cryptography` library. **AES-256-GCM, streamed in chunks**, key derived from a passphrase:

- Passphrase from env `MAGPIE_BACKUP_PASSPHRASE` — **out-of-band** (password manager), **distinct** from `MAGPIE_TOKEN_ENCRYPTION_KEY` (D-78). The command **refuses to run** if it's unset.
- KDF: **scrypt** (`cryptography.hazmat.primitives.kdf.scrypt`) with a random 16-byte salt → 32-byte key.
- Cipher: **AES-256-GCM** (`AESGCM`), processing the tar in fixed chunks (e.g. 1 MiB), each chunk with its own 12-byte nonce; write a small header (magic, version, salt) then length-prefixed `(nonce, ciphertext+tag)` records. Decrypt verifies every tag (tamper-evident).
- (This refines D-77's "age/gpg" *illustration* toward the existing dependency — the **requirement** is unchanged: client-side encryption, passphrase out-of-band. `age` / `gpg` remain acceptable operator alternatives, noted in the runbook; the built-in path needs no Windows binary install. Flagged for review, not a silent change.)

Tests: encrypt→decrypt round-trip equals the original bytes; a single flipped byte makes decryption **fail loudly** (GCM auth).

---

4. Out-of-band secrets (D-78) — the bit that makes

“restore” honest

The archive holds the eBay credential **as Fernet ciphertext** (it lives in the DB). The key that decrypts it (`MAGPIE_TOKEN_ENCRYPTION_KEY`) is **not** in the archive. So a restore to a *new* machine yields the credential row but cannot use it until either the key is supplied from the password manager **or** eBay is reconnected via the 2-minute paste-back flow. The runbook states this explicitly. **Never** place `MAGPIE_TOKEN_ENCRYPTION_KEY` , the eBay Cert ID/secret, `MAGPIE_BACKUP_PASSPHRASE` , or the cloud credentials inside the archive.

---

5. The restore command (`manage.py restore`)

`python manage.py restore <archive.enc> --target <dir>` :

1. Decrypt (needs `MAGPIE_BACKUP_PASSPHRASE`) → tar → unpack to a temp area.
2. Place the SQLite snapshot at `<target>\db.sqlite3` and the media tree at `<target>\media_files` (**clean target** — refuse to overwrite a non-empty target unless `--force`).
3. Print restored row counts for the key tables (items/photos/valuations/drafts/credential) for verification.
4. Print the eBay-reconnect reminder (key-from-password-manager or paste-back). Restore must **not** touch the live DB unless the operator explicitly points `--target` at it.

---

6. Scheduling (Windows) + retention

- A wrapper `scripts\backup.ps1` : activates the venv, sets working dir, runs `python manage.py backup --upload` . Quote all paths (the install path and `media_files` contain spaces — Windows gotcha).
- Register with **Task Scheduler** (daily): provide the exact `schtasks /create /tn "Magpie Backup" /tr "pwsh -File <path>\scripts\backup.ps1" /sc daily /st 02:00 /rl LIMITED` (and a documented GUI alternative). Runs unattended under Regan’s account.
- **Retention:** keep `--keep-daily 7 --keep-weekly 4` locally; prune older `.enc` files. `rclone` mirrors/keeps remote copies (document the remote retention choice; don’t auto-delete remote in S8).

- **Linux equivalent (documented for the future migration, not built):** the same command under a `systemd` timer.

---

7. Logging (D-80)

- Configure Django `LOGGING` with a **`RotatingFileHandler`** → `…\backend\logs\magpie.log` (e.g. 5 MB × 5 backups), plus console. App + Django + the eBay/audit loggers route here.
- Carry the structural no-secret discipline: tokens/codes/secrets never logged (already the rule; confirm no stray `print` / `logger.debug` leaks in the eBay paths).
- **One-time secret-marker grep** over the real log file (markers: `gAAAA`, `v^1`, `code=`, `refresh_token`, `client_secret`, the key/passphrase names) → result recorded in the evidence file. This formally closes the first-flight log-scan limitation.

---

8. Acceptance criteria (D-79 — operational evidence, headline = tested restore)

CI stays green (no regressions). Closure also requires, in **`RESTORE_DRILL_EVIDENCE.md`**:

1. `manage.py backup` runs and produces a `.enc` archive.
2. Archive is encrypted (header present; not a readable tar).
3. Archive contains the SQLite snapshot + media tree + `env_manifest.json` (names only) + runbook.
4. `manage.py restore` onto a **clean** target succeeds.
5. The app boots against the restored DB/media.
6. Row counts match the source (8/2/5/2/1 at time of writing) and photos render.
7. eBay reconnect path proven or explicitly rehearsed (key-from-manager or paste-back).
8. `logs\magpie.log` exists, rotates, and **passes the no-secret grep**.
9. **The scheduled Task Scheduler job fires unattended at least once and uploads** (proof: a fresh `.enc` on the cloud remote with a timestamp from the unattended run).

10. The evidence file records all of the above.

---

9. Test plan (these run in CI on SQLite — so the logic is covered even though the *drill* is manual)

- Backup command: produces an archive; archive decrypts; contains DB snapshot + media + manifest + runbook; `env_manifest.json` has **names only** (assert no value-looking strings); refuses without `MAGPIE_BACKUP_PASSPHRASE`.
- Encryption: round-trip equality; tamper (1 byte) → decryption raises.
- Restore: into a temp dir → SQLite + media placed; **row counts equal** a freshly-seeded source; refuses a non-empty target without `--force`.
- Logging: handler writes + rotates; a crafted log line containing a fake token is **not** what the secret-grep would pass (verify the grep catches markers).
- No new migrations expected; `manage.py backup` / `restore` covered; CI "PostgreSQL Validation" stays green (it exercises the code paths on its DB; the backup/restore logic is DB-agnostic for these tests or runs against a temp SQLite file).

---

10. Out of scope (Sprint 8)

Postgres runtime migration (separate Sprint 9 decision) · any deployment/runtime change (gunicorn/Waitress/proxy/static serving — Sprint 9) · `DEBUG=False` /final `ALLOWED_HOSTS` /secure cookies (Sprint 9) · HTTPS / internet exposure (documented runbook only) · remote-retention automation/lifecycle rules beyond a documented choice · backup-management UI · any application feature.

---

11. Regan prerequisites

1. **Pick a cloud remote** and run a one-time `rclone config` for it (Backblaze B2 is cheap + `rclone`-native; S3/Drive also fine). Credentials entered locally, never in chat.
2. **Set** `MAGPIE_BACKUP_PASSPHRASE` in the local env and **store it in your password manager** alongside `MAGPIE_TOKEN_ENCRYPTION_KEY` and the eBay Cert ID — these four are the out-of-band secrets that are never in a backup.

3. Install `rclone` (portable Windows binary on PATH) and confirm `age` / `gpg` are *not* required.

Hand-off notes

- **A backup you have never restored is not a backup.** The tested restore (criteria 4–6) is the point of the sprint; everything else serves it.
- Use the SQLite **online backup API**, never a bare file copy of a live DB.
- The token key and the backup passphrase are **different secrets** and **neither** goes in the archive.
- Quote every Windows path (spaces in the install path + `media_files`).
- Do not migrate to Postgres and do not change how the app runs — that is Sprint 9. If you find yourself editing how Magpie *serves*, you have left the sprint.
- When ambiguous, prefer the smaller implementation that satisfies §8.